

<과제>

2022012408

염재민

1. 순차적으로 실행되는 구문만을 가지는 소프트웨어 언어와 병렬 실행이 추가된 하드웨어 언어의 차이

:일반적으로 많이 사용하는 소프트웨어 언어인 C와 C++등은 순서에 따라 하나씩 실행하는 것을 기본으로한다.

반면 Verilog와 같은 하드웨어 기술 언어(HDL)는 실제 하드웨어의 동작을 표현하기에 여러 회로가 동시에 동작하는 병렬 실행을 기본으로 한다.

따라서 소프트웨어에서는 if, for 등의 문장이 주로 실행 순서와 알고리즘을 나타내지만 하드웨어 언어에서는 같은 구문이라도 어떤 회로를 만들 것인지를 나타내는 의미가 추가됨.

2. 시현장치(CRT, 디스플레이 패널)의 비월 주사에 대한 조사, 화소점을 순차적으로 제어하기 위해 VSYNC, HSYNC 신호의 역할 조사

: 비월주사를 정리하기 전 순차주사(Progressive Sacn)을 짚고 가자. 순차주사란 말그대로 1,2,3,~,8 순서대로 위에서 아래로 순서대로 그리는 것이다.

반면 비월 주사는 한화면을 두 번에 나누는데, 첫 번째(홀수 필드): 1, 3, 5, 7

두 번째(짝수 필드):2, 4, 6, 8 로 두 번 나누어 표시하는거다. 후에 두 개를 합치면 우리가 보는 하나의 프레임으로 구성이됨.

VSYNC는 수직 동기 신호, HSYNC는 수평 동기 신호임.

VSYNC는 새로운 화면의 시작을 맞추고 HSYNC는 새로운 줄의 시작을 맞춘다고 생각.

1번째 줄	—————→	HSYNC
2번째 줄	—————→	HSYNC
3번째 줄	—————→	HSYNC
4번째 줄	—————→	HSYNC
5번째 줄	—————→	HSYNC
6번째 줄	—————→	HSYNC

...

마지막 줄 —————→

VSYNC



다음 화면

쉬운 이해를 위하여 위를 참고하면 좋다.

3. 위 교수님 자료블로그 탁구게임기 소스코드 읽어보기

:(정보가 부족하여 추후 추가하여 제출하겠습니다.)

4. 소프트웨어로도 충분히 구현 가능함에도 불구하고 굳이 전용 칩(하드웨어)이 필요한 이유 생각해보기

: 탁구게임기처럼 단순한 기능은 고속 마이크로컨트롤러를 사용하면 충분히 구현이 가능하다. 그럼에도 전용 칩을 만드는 이유는 병렬철와 전력 효율, 그리고 실시간 처리 능력에 있다. 소프트웨어 방식의 경우, 순서대로 작업을 처리하는 반면, 하드웨어에서는 각각의 기능을 회로로 만들어 동시에 동작을 시킬 수 있다. 따라서 같은 작업을 반복해야하는 경우에는 전용 하드웨어를 통해 불필요한 연산과 프로그램 실행 횟수를 효율적으로 줄일 수 있다. 또한 하드웨어를 병렬구조로 설계시, 낮은 클럭에서도 필요한 처리량을 확보가능하여 전력소모를 줄이는데 유리하다. 다만 장점이 명확하듯 단점도 확실하다. 칩 설계 과정에서 많은시간과 비용이 소모되고 기능변경이 어렵다는 단점이있다. 따라서 모든 프로그램을 전용 칩으로 만드는 것이 좋은 것만이 아니며 각각의 장단점을 살려줄수있는경우에 맞게 설계해야한다.

5. ++yPos와 yPos++의 차이에 대하여 프로그래밍 언어의 순차실행 구문의 관점에서 설명해 보라. 하드웨어 기술 언어(베릴로그)의 병렬 구문 실행과 비교해보자. 아울러 베릴로그 하드웨어 언어에서 단항 연산자 ++, -- 등을 둘 경우 그 위험성(?)을 설명해 보자.

: 둘다 기능적으로는 yPos의 값을 1증가시키는 것이다. 차이는 증가한 값을 언제 사용하는가이다.

++yPos: 전위증가

yPos++: 후위증가

말그대로 ++yPos의 경우는 먼저 증가시켜 사용하고 yPos++는 먼저 사용하고 나중에 증가시킨다.

C와 C++ 같은 소프트웨어에서는 프로그램이 문장의 실행순서를 따라가는걸 생각해보면

++가 앞에 있냐 뒤에 있느냐가 실제 결과에 영향을 준다.

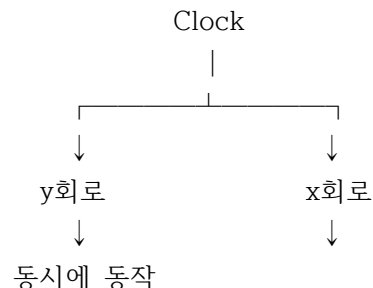
반면 Verilog와 같은 하드웨어 기술 언어에서는 여러 하드웨어 블록이 병렬적으로 동작하므로 소프트웨어와 같은 순차적인 실행 순서로만 생각할 수 없다.

C/C++ 같은 구조의 경우는 첫 번째 always 실행



두 번째 always 실행 처럼 순차진행되지만,

베릴로그같은 하드웨어는 병렬로 동시에 같은 클럭이 일어날수있는거다.



특히 여러 블록에서 같은 신호를 동시에 변경하거나 읽을 경우 의도하지 않은 동작이나 시뮬레이션과 실제 하드웨어 사이의 차이가 발생할 수 있다. 따라서 Verilog에서는 $yPos \leq yPos + 1$ 과 같이 하드웨어의 동작을 명확하게 표현하는 방법이 더 안전하다.

↑(다음 클럭에서 yPos를 현재 값보다 1 증가시킨다)